Comprehensive Technical Report

A Scalable MLOps Ecosystem for Predictive Industrial Analytics

Name	Sri Mahaguhan S (DA25S021)

1. Abstract

This report presents a robust, end-to-end MLOps framework designed for the **predictive maintenance of industrial assets**. The system transitions from a static model to a dynamic service by integrating **Data Version Control (DVC)**, **MLflow experimentation**, and a **Prometheus/Grafana observability stack**. The solution is fully containerized using Docker Swarm/Compose to ensure environment parity. The resulting model achieves an accuracy of **98.25%**, served through a decoupled FastAPI backend and Streamlit frontend.

2. Infrastructure & Orchestration *(Docker & Airflow)*

The system is built on a **microservices architecture** to satisfy the Loose Coupling requirement.

- **Dockerization** — Every component — from the Postgres database to the Grafana dashboard — is isolated within its own container. A multi-stage build process in `api.Dockerfile` was utilized to optimize image size and security.
- **Environment Parity** — By using a Debian-based Python 3.11 environment across all services, we eliminated *"it works on my machine"* inconsistencies, ensuring the macOS development environment perfectly mirrors the production Linux kernel.
- **Orchestration** — Apache Airflow serves as the workflow orchestrator. A custom DAG manages the data ingestion pipeline, fetching raw sensor data from Indian government portals and processing it into a training-ready state.

3. Data Integrity & Reproducibility *(DVC)*

To solve the problem of **"Data Drift"** and **"Training-Serving Skew,"** Data Version Control (DVC) was implemented.

- **Tracking** — Large datasets and model artifacts are not stored in Git. Instead, DVC generates lightweight `.dvc` pointer files.
- **DAG Visualization** — `dvc dag` was utilized to visualize the dependency graph between data ingestion, feature engineering, and model training.
- **Reproducibility** — Every successful model run is tied to a specific Git commit hash, allowing the team to revert to any previous state of the data and model with **100% fidelity**.

4. Experimentation & Model Lifecycle *(MLflow)*

The model development phase was managed via MLflow, serving as the central **Model Registry** and tracking server.

4.1 Hyperparameter Logging

Multiple runs were conducted, ultimately selecting a **Random Forest Classifier** with the following optimized parameters:

Parameter	Value
`n_estimators`	100
`max_depth`	10
`random_state`	42

4.2 Performance Metrics

The model exhibits high reliability, as evidenced by the logged metrics:

Metric	Value
Accuracy	0.9825
Precision *(Macro)*	0.9054
Recall *(Macro)*	0.7687
F1-Score *(Macro)*	0.8222

Model Signature: A strict schema was enforced to ensure the model only accepts the specified **5 sensor inputs** *(Air Temp, Process Temp, Speed, Torque, Tool Wear)*.

---

5. Inference & User Experience *(FastAPI & Streamlit)*

The serving layer is designed for **high throughput** and **user accessibility**.

FastAPI Backend

Acts as the inference engine with the following capabilities:

Feature	Details
Endpoints	`/predict` for classification; `/health` and `/ready` probes for automated monitoring.
Validation	Pydantic rejects malformed inputs before they reach the model.

Streamlit Frontend

Designed for non-technical factory operators, featuring:

- **Real-time Simulation** — Sliders to input sensor data.
- **Foolproof UX** — Color-coded alerts *(Green for Healthy, Red for Failure)* to facilitate immediate decision-making.

---

6. Observability & Alerting *(Prometheus, Grafana, AlertManager)*

A critical pillar of this project is its **Near-Real-Time (NRT)** monitoring capability.

6.1 Prometheus Instrumentation

The FastAPI backend is instrumented to export custom metrics to Prometheus:

Metric	Type	Description
`api_predictions_total`	Counter	Tracks total requests, labeled by result (`success` or `error`).
`api_inference_duration_seconds`	Histogram	Tracks the latency of the model.
`data_drift_score`	Gauge	Tracks the statistical shift in input features.

6.2 Grafana Visualization

A Grafana dashboard was designed to provide a **single pane of glass** for:

- **Inference Throughput** — Real-time requests per second.
- **Failure Distribution** — A pie chart showing the types of failures detected *(e.g., Tool Wear vs. Power Failure)*.
- **System Health** — CPU and RAM usage of the host machine via Node Exporter.

6.3 AlertManager Logic

A proactive alerting system was established in `alert_rules.yml`:

Alert	Trigger Condition
High Error Rate	`>5%` of predictions result in an error over a 5-minute window.
Inference Latency Anomaly	95th percentile of requests takes `>0.5s`.
Data Drift	Drift score exceeds a threshold of `0.25`, alerting the engineer to initiate a retraining cycle.

---

7. Conclusion & Future Work

This project demonstrates a **production-grade MLOps maturity level**. By integrating DVC, Docker, and the Prometheus/Grafana stack, we have created a system that is not only accurate but also **observable** and **reproducible**.

Future work will focus on implementing **Continuous Training (CT)**, where the Airflow DAG is automatically triggered by the Prometheus *"Data Drift"* alert — closing the loop of the ML lifecycle.